

Cloud Data Platform

Guide complet expliqué simplement Phase 1 — Landing Zone AWS + Kubernetes

Auteur	Mohammed Dahiry — Data Engineer
Projet	PFE 2025–2026
Région AWS	us-east-1
State backend	Cloudflare R2 (S3-compatible)
Scope	Phase 1 — Landing Zone (fondation complète)

Ce document explique le projet Cloud Data Platform comme si tu avais 12 ans. Pas de jargon non expliqué. Chaque concept = une analogie simple.

C'est quoi tout ça ?

Avant de toucher à du code, on va comprendre le problème qu'on résout et pourquoi on a besoin de toute cette infrastructure.

Le problème de base

Imagine une grande entreprise avec des milliers d'employés. Chaque département a ses propres fichiers Excel, ses propres bases de données, et personne ne peut voir les données des autres. C'est ce qu'on appelle des **silos de données**.

En plus, quand quelqu'un veut analyser les ventes de l'année, il doit appeler 5 personnes différentes pour récupérer les fichiers, les assembler à la main, et souvent les données sont déjà obsolètes quand il termine.

■ L'analogie parfaite

Imagine que tu veux faire une tarte aux pommes. Les pommes sont dans un frigo verrouillé au sous-sol, la farine est dans un placard dont seul le cuisinier a la clé, et le four appartient à un voisin. Notre projet, c'est de construire une cuisine commune où tout est accessible, organisé, et disponible 24h/24.

La solution : une plateforme de données moderne

Notre projet construit une **Data Platform** — une infrastructure centralisée qui :

- Collecte les données de toutes les sources (bases de données, fichiers Excel, flux en temps réel)
- Les stocke de manière organisée et sécurisée
- Les transforme pour qu'elles soient propres et utilisables
- Les expose aux analystes et aux outils BI comme Power BI

L'architecture en 7 couches

Comme un gâteau à étages, notre plateforme est organisée en 7 couches superposées. Chaque couche a un rôle précis et dépend de la couche en dessous.

	Couche	Technologies	Rôle
1	■ ■ Sources	PostgreSQL, SQL Server, Informix, CSV, Excel	Les données brutes à la source
2	■ Ingestion	NiFi, Kafka, Spark Streaming	Aspirer les données vers la plateforme

3	■■■ Stockage	MinIO + Apache Iceberg + Hive Metastore	Garder les données organisées
4	■■■ Calcul	Spark, dbt, Airflow, JupyterHub	Nettoyer et transformer les données
5	■ Virtualisation	Dremio Nautilus	Requêter sans déplacer les données
6	■ Présentation	Power BI	Afficher les graphiques et tableaux de bord
7	■■■ Observabilité	Prometheus, Grafana, OpenObserve	Surveiller que tout fonctionne

■ Ce document couvre la Phase 1

La Phase 1 est la FONDATION. C'est comme construire les fondations d'une maison avant de poser les murs. Sans fondations solides, tout le reste s'effondre. On va créer le réseau, la sécurité, les serveurs, et l'orchestrateur Kubernetes.

CHAPITRE 2

Les mots compliqués expliqués simplement

Avant de continuer, voici un lexique des termes techniques que tu vas rencontrer partout. Lis ce chapitre comme un dictionnaire — reviens y quand tu bloques sur un mot.

AWS	<i>Amazon Web Services</i>
Le loueur de serveurs. Plutôt que d'acheter tes propres ordinateurs, tu loues des serveurs chez Amazon à la minute. C'est comme louer une voiture au lieu d'en acheter une.	
EKS	<i>Elastic Kubernetes Service</i>
Le service AWS qui gère Kubernetes pour toi (voir Kubernetes ci-dessous).	
Kubernetes (K8s)	<i>Orchestrateur de conteneurs</i>
Imagine un chef d'orchestre qui dit à chaque musicien (conteneur) quand jouer, combien de musiciens sont nécessaires, et remplace automatiquement ceux qui tombent malades.	
Terraform	<i>Infrastructure as Code</i>
Au lieu de cliquer sur des boutons dans la console AWS, tu écris du code qui décrit ce que tu veux. Terraform lit ce code et crée tout automatiquement. C'est comme donner une recette à un robot-cuisinier.	
VPC	<i>Virtual Private Cloud</i>
Ton réseau privé sur AWS. Comme si Amazon te donnait un appartement dans un immeuble géant — tu as tes propres murs, clés et règles de sécurité.	
MinIO	<i>Stockage objet S3-compatible</i>
Un serveur de stockage qui ressemble à Google Drive mais pour les données massives. Compatible avec le protocole S3 d'Amazon.	
Iceberg	<i>Format de table open source</i>
Un format de fichier intelligent pour les grandes tables de données. Il supporte les transactions, le versioning, et l'évolution de schéma.	

IAM / IRSA	<i>Gestion des droits AWS</i>
Le système qui contrôle qui peut faire quoi. IRSA = chaque service Kubernetes a sa propre identité AWS avec des droits minimaux.	
OIDC	<i>Protocole d'authentification</i>
Comme le bouton 'Se connecter avec Google' — GitHub Actions prouve son identité à AWS sans avoir de mot de passe.	
KMS	<i>Key Management Service</i>
Le coffre-fort d'AWS. Toutes les données sont chiffrées avec des clés que seul toi (et les services autorisés) pouvez utiliser.	
Cloudflare R2	<i>Stockage objet gratuit</i>
Le service de stockage de Cloudflare, compatible S3, gratuit jusqu'à 10 Go/mois. On l'utilise pour stocker l'état Terraform.	
NAT Gateway	<i>Passerelle réseau</i>
Permet à tes serveurs privés (sans IP publique) d'accéder à Internet pour télécharger des mises à jour, sans être exposés directement.	
GitHub Actions	<i>CI/CD automatisé</i>
Chaque fois que tu pousses du code sur GitHub, une série d'actions se déclenche automatiquement : tests, déploiement, etc.	
RBAC	<i>Role-Based Access Control</i>
Contrôle des accès par rôle. Comme dans un hôpital : le médecin accède aux dossiers médicaux, l'agent d'entretien n'y accède pas.	

CHAPITRE 3

La structure du projet

Voyons comment le code est organisé dans le dépôt GitHub. Comprendre la structure avant de coder, c'est comme lire le plan d'une maison avant de poser les briques.

Arborescence du dépôt

Voici comment les fichiers sont organisés et pourquoi :

terraform/	Tout le code d'infrastructure AWS
bootstrap/	Scripts à exécuter UNE SEULE FOIS au tout début
modules/vpc/	Code pour créer le réseau (VPC, subnets, NAT)
modules/kms/	Code pour créer les clés de chiffrement
modules/iam/	Code pour créer les droits et rôles AWS
modules/eks/	Code pour créer le cluster Kubernetes
modules/r2-backend-bootstrap/	Code pour créer le bucket R2 sur Cloudflare
environments/dev/	La configuration de l'environnement de développement
bootstrap/	Manifests Kubernetes (namespaces, RBAC, NetworkPolicy)
scripts/	Scripts bash utiles
docs/	Documentation technique
.github/workflows/	Pipelines CI/CD GitHub Actions

Le principe des modules Terraform

■ Modules = briques LEGO réutilisables

Chaque module fait une seule chose et la fait bien. Le module VPC crée le réseau. Le module EKS crée le cluster. L'environnement 'dev' assemble ces briques comme on assemble un château LEGO. Si demain on veut un environnement 'prod', on réutilise les mêmes briques avec des paramètres différents.

Chaque environnement (dev, prod) contient 3 fichiers clés :

main.tf	Liste les modules à utiliser et les connecte entre eux
variables.tf	Déclare les paramètres configurables (taille des serveurs, région, etc.)
terraform.tfvars	Donne les valeurs réelles des paramètres pour cet environnement

Étape 0 — Le bootstrap (une seule fois)

Avant de tout lancer, il faut préparer deux choses : le bucket R2 qui va stocker l'état Terraform, et les droits AWS pour GitHub Actions. Cette étape ne s'exécute qu'une seule fois dans la vie du projet.

Pourquoi stocker l'état Terraform ?

Terraform a besoin de se souvenir de ce qu'il a créé. Il stocke ça dans un fichier appelé **terraform.tfstate**. Si ce fichier est sur ton ordinateur et que ton collègue modifie l'infrastructure, il y aura des conflits catastrophiques.

■ ■ Solution : stocker l'état dans le cloud

On stocke le fichier d'état sur Cloudflare R2 (un stockage en ligne). Ainsi, tout le monde partage le même état, et Terraform bloque les modifications simultanées. R2 est gratuit jusqu'à 10 Go/mois et indépendant d'AWS (si ton compte AWS a un problème, ton état est toujours accessible).

Étape 4.1 — Créer le bucket R2 Cloudflare

Dans ton terminal, depuis le dossier terraform/bootstrap/ :

```
# 1. Aller dans le dossier bootstrap
cd terraform/bootstrap/

# 2. Créer un fichier avec tes credentials Cloudflare
# (récupère-les sur dash.cloudflare.com > R2 > Manage R2 API tokens)
export CF_ACCOUNT_ID='ton-account-id'
export CF_API_TOKEN='ton-api-token'

# 3. Initialiser et appliquer
terraform init
terraform apply

# Résultat : bucket 'terraform-state-pfe' créé sur R2
```


Étape 4.2 — Configurer OIDC GitHub Actions → AWS

On veut que GitHub Actions puisse créer des ressources AWS sans stocker de mots de passe (clés AWS) dans GitHub. La solution : OIDC — GitHub prouve son identité à AWS avec un token temporaire.

■ Analogie : badge temporaire

C'est comme si GitHub Actions se présentait à l'entrée d'AWS avec une lettre d'introduction signée par Anthropic (l'autorité de confiance). AWS vérifie la signature, est convaincu, et émet un badge temporaire (15 minutes) sans jamais donner les clés permanentes à GitHub.

```
# Dans terraform/bootstrap/, un fichier oidc.tf crée :

# - Un provider OIDC AWS qui fait confiance à GitHub

# - Un rôle IAM que GitHub Actions peut 'assumer'

# - Les politiques minimales nécessaires (terraform plan/apply)

# Dans le workflow GitHub Actions (.github/workflows/terraform.yml) :

# permissions:

# id-token: write <- permet à GH Actions de générer un token OIDC

# contents: read

# uses: aws-actions/configure-aws-credentials@v4

# with:

# role-to-assume: arn:aws:iam::ACCOUNT:role/github-actions-role

# aws-region: us-east-1
```

Le réseau — VPC et sous-réseaux

Le réseau est la fondation de tout. Avant de créer des serveurs, il faut construire les routes et les clôtures qui vont les entourer.

C'est quoi un VPC ?

Un **VPC (Virtual Private Cloud)** est ton espace réseau privé chez AWS. Pense à un immeuble de bureaux : AWS possède l'immeuble, toi tu loues un étage entier. Ton étage a ses propres portes, ses propres règles d'accès, et les visiteurs extérieurs ne peuvent pas y entrer directement.

Notre VPC : 3 zones de disponibilité

Notre VPC s'étend sur **3 zones de disponibilité (AZ)** dans us-east-1 : us-east-1a, us-east-1b, us-east-1c. Une zone = un datacenter physiquement séparé. Si un datacenter tombe en panne, les autres continuent de fonctionner.

Subnet	Type	AZ	Accès Internet	Contient
public-1a/1b/1c	Public	3 AZs	Direct (IGW)	Load Balancers, NAT GW
private-1a/1b/1c	Privé	3 AZs	Via NAT GW	Nœuds EKS, Pods Kubernetes

Le NAT Gateway : comment les serveurs privés accèdent à Internet

■ Analogie : boîte aux lettres partagée

Tes serveurs privés n'ont pas d'adresse Internet directe (comme une maison sans numéro). Le NAT Gateway est comme une boîte aux lettres au bout de la rue : tes serveurs envoient leur courrier là, le NAT le fait suivre vers Internet, et ramène les réponses. Les gens d'Internet ne connaissent jamais l'adresse réelle de tes serveurs.

■■ Choix MVP : un seul NAT Gateway

En production, on aurait un NAT par zone de disponibilité (3 au total) pour la haute disponibilité. En MVP, on en a un seul (dans us-east-1a) pour économiser de l'argent. C'est documenté comme un SPOF (Single Point of Failure) connu et acceptable pour ce contexte.

Les Security Groups : les pare-feux des serveurs

Chaque serveur a un **Security Group** — une liste de règles qui dit qui peut lui parler et sur quel port. Les nœuds EKS n'acceptent que le trafic venant d'autres nœuds EKS et du control plane AWS. Aucun port n'est ouvert vers Internet directement.

La sécurité — KMS et IAM

La sécurité n'est pas une option qu'on ajoute à la fin. Dans notre projet, elle est construite dès la fondation. Deux piliers : le chiffrement (KMS) et les droits d'accès (IAM).

KMS : toutes les données chiffrées

Toutes les données stockées (disques EBS, secrets, logs) sont chiffrées avec des **clés KMS gérées par le client (CMK)**. C'est comme si chaque tiroir de ton bureau avait sa propre serrure unique, et toi seul as les clés.

Alias KMS	Protège quoi
alias/dataplatform-ebs-dev	Disques EBS des nœuds Kubernetes
alias/dataplatform-secrets-dev	Secrets stockés dans AWS Secrets Manager
alias/dataplatform-logs-dev	Logs CloudWatch des workloads

IAM et IRSA : le principe du moindre privilège

Chaque service Kubernetes qui a besoin d'accéder à AWS reçoit son propre rôle IAM avec **uniquement les droits nécessaires**. Rien de plus. C'est le principe du moindre privilège.

ServiceAccount K8s	Rôle IAM	Droits accordés
cluster-autoscaler	eks-cluster-autoscaler-role	Lire/modifier les AutoScaling Groups
ebs-csi-controller	ebs-csi-controller-role	Créer/supprimer des volumes EBS
external-secrets	external-secrets-role	Lire les secrets dans Secrets Manager
aws-load-balancer	load-balancer-controller-role	Gérer les Load Balancers AWS

■ Comment IRSA fonctionne

1. Un Pod Kubernetes démarre avec un ServiceAccount (ex: ebs-csi-controller).
2. Kubernetes injecte automatiquement un token JWT dans le Pod.
3. Le Pod envoie ce token à AWS STS pour obtenir des credentials temporaires.
4. AWS vérifie que le token vient bien de notre cluster EKS (via OIDC).
5. AWS émet des credentials qui expirent après 1 heure maximum. Résultat : aucune clé AWS stockée nulle part dans le code.

Kubernetes (EKS) — Le chef d'orchestre

Kubernetes est le cœur de notre infrastructure. Il gère automatiquement où et comment tournent nos applications. Comprendre Kubernetes est essentiel pour tout le reste.

C'est quoi Kubernetes ?

Kubernetes (abrégé K8s) est un système qui gère automatiquement les applications conteneurisées. Au lieu de se connecter manuellement à chaque serveur pour démarrer/arrêter des programmes, tu décris ce que tu veux et Kubernetes le fait.

■ Analogie : chef d'orchestre

Imagine un orchestre de 100 musiciens (tes applications). Sans chef, c'est le chaos. Le chef d'orchestre (Kubernetes) sait quel musicien joue quelle partition, remplace immédiatement un musicien malade, ajoute des musiciens si la musique devient trop complexe, et retire des musiciens si la salle est vide. Toi, tu donnes juste la partition (le manifest YAML).

Notre cluster EKS

Version K8s	1.30
Type	EKS managed (AWS gère le control plane)
Région	us-east-1
API server	Endpoint privé + allowlist d'IPs publiques
Logging	API server, audit, authenticator, scheduler → CloudWatch

Les deux node groups

Nos applications tournent sur des **nœuds (nodes)** — des machines EC2 qui exécutent les Pods. On a deux groupes de nœuds avec des comportements différents :

	stateful-ng	compute-ng
Workloads	MinIO, PostgreSQL, Hive Metastore, Kafka	Spark, dbt, Airflow workers
Toujours allumé ?	Oui (MinSize=1)	Non (scale à 0 la nuit)
Type EC2	m5.xlarge (4 vCPU, 16 GB)	m5.2xlarge (8 vCPU, 32 GB)

Taille min/max	1 – 3 nœuds	0 – 10 nœuds
Disque	EBS gp3 100 GB chiffré	EBS gp3 100 GB chiffré
Pourquoi séparé ?	Ne jamais perdre les données	Économiser €€ hors heures

Les namespaces : organisation interne de Kubernetes

Dans Kubernetes, les **namespaces** sont des espaces logiques isolés. C'est comme des dossiers dans un ordinateur : chaque équipe range ses affaires dans son dossier, avec ses propres règles d'accès.

Namespace	Contenu	Équipe
platform-ingestion	NiFi, Kafka, Kafka Connect	Data Engineering
platform-storage	MinIO Operator, MinIO Tenant, PostgreSQL (sources)	Infra
platform-metastore	Hive Metastore, PostgreSQL (HMS)	Data Engineering
platform-compute	Spark Operator, dbt jobs	Data Engineering
platform-orchestr	Airflow, JupyterHub	Data Engineering
platform-serving	Dremio	Analytics
platform-monitoring	Prometheus, Grafana	SRE
platform-logging	Fluent Bit, OpenObserve	SRE
platform-security	Cloudflare Operator, cert-manager	Infra

Le pipeline CI/CD — GitHub Actions

Chaque modification du code doit passer par un processus automatisé de validation et déploiement. C'est le CI/CD — l'artère principale du projet.

C'est quoi CI/CD ?

CI (Continuous Integration) = chaque fois que tu pousses du code, des tests s'exécutent automatiquement pour vérifier que rien n'est cassé. **CD** (Continuous Deployment) = si les tests passent et que c'est sur la branche main, le code est déployé automatiquement en production.

■ Analogie : chaîne de montage automobile

Tu soudes une pièce (tu modifies du code). La pièce passe automatiquement par une batterie de tests qualité (CI). Si tout est OK, la voiture avance sur la chaîne et la pièce est installée (CD). Si un test échoue, la chaîne s'arrête et tu reçois une alerte. Zéro intervention manuelle entre 'coder' et 'déployer'.

Notre pipeline en 5 étapes

1 — Lint & Validate	<code>terraform fmt, terraform validate, tflint</code>
Vérifie que le code est syntaxiquement correct. Comme la vérification orthographique avant d'envoyer un email.	
2 — Plan (sur Pull Request)	<code>terraform plan</code>
Calcule et affiche les changements que Terraform va faire, sans rien modifier. Posté en commentaire sur la PR pour review.	
3 — Apply (après merge sur main)	<code>terraform apply -auto-approve</code>
Applique réellement les changements sur AWS. Seulement déclenché quand le code arrive sur la branche principale.	
4 — Helm Deploy	<code>helm upgrade --install</code>
Déploie ou met à jour les applications Kubernetes (MinIO, Kafka, Spark, etc.) après les changements d'infrastructure.	

5 — Smoke Test

Tests de connectivité basiques

Vérifie rapidement que les services répondent après le déploiement.

Comment lancer le projet pour la première fois

Séquence à suivre dans l'ordre exact, une seule fois au démarrage du projet :

```
# ÉTAPE 1 : Bootstrap (UNE SEULE FOIS)

cd terraform/bootstrap/

terraform init && terraform apply

# → Crée le bucket R2 + OIDC provider AWS pour GitHub

# ÉTAPE 2 : Initialiser l'environnement dev

cd ../environments/dev/

terraform init # télécharge les providers, configure le backend R2

terraform plan # voir ce qui va être créé

terraform apply # créer VPC + KMS + IAM + EKS

# ÉTAPE 3 : Appliquer les manifests Kubernetes

cd ../../..

kubectl apply -f bootstrap/namespaces/

kubectl apply -f bootstrap/rbac/

kubectl apply -f bootstrap/network-policies/

kubectl apply -f bootstrap/storage-classes/
```


Bonnes pratiques appliquées

Ce projet ne se contente pas de 'fonctionner'. Il est conçu pour être maintenable, sécurisé, et évolutif. Voici les principes d'ingénierie appliqués et pourquoi ils comptent.

■ Infrastructure Immuable

Tout est défini en code Terraform. Aucun changement manuel en production. Si tu veux modifier quelque chose, tu modifies le code, tu crées une PR, tu fais reviewer, et le pipeline CI/CD applique. Résultat : tout est traçable, reproductible, et réversible.

■ Least Privilege IAM

Chaque service a les droits minimaux nécessaires via IRSA. Le Pod Spark peut lire/écrire dans MinIO, mais pas modifier les Security Groups. Si un Pod est compromis, l'attaquant ne peut rien faire d'autre.

■ Secrets hors du code

Aucun mot de passe, clé API, ou token dans le code Git. Tout est dans AWS Secrets Manager. L'External Secrets Operator les injecte automatiquement dans les Pods au démarrage.

■ Cloud Agnostique

MinIO (pas S3 natif), Iceberg (pas Delta Lake propriétaire), Kubernetes (pas ECS/Fargate). Pour passer de AWS à un cloud on-premise, tu changes EBS → Ceph RBD et rien d'autre dans le code applicatif.

■ Version Pinning strict

Toutes les versions sont fixées : Terraform 1.7.0, EKS 1.30, MinIO Operator 6.0.0, Kafka 3.7.0. Si tu réinstalle dans 6 mois, tu obtiens exactement le même résultat.

■ Cost Control dès le départ

Lambda scheduling : workers EKS scale à 0 la nuit (18h → 8h). Un seul NAT Gateway en MVP. Billing alerts AWS configurés dès J1. EKS control plane reste actif (\$0.10/h) — seuls les workers EC2 scalent.

Checklist de démarrage et suite du projet

Tout ce qu'il faut cocher avant de dire 'Phase 1 terminée', et un aperçu des 6 phases suivantes.

Checklist Phase 1 — Tout cocher avant de passer à la phase 2

	Élément à vérifier	Comment vérifier
■	Bucket R2 Cloudflare créé et accessible	terraform/bootstrap/ appliqué
■	Provider OIDC AWS configuré pour GitHub Actions	terraform/bootstrap/ appliqué
■	VPC créé avec 3 AZs, subnets publics/privés, NAT Gateway	terraform/environments/dev/ appliqué
■	3 clés KMS créées (EBS, secrets, logs)	module/kms/ appliqué
■	Rôles IRSA créés (cluster-autoscaler, ebs-csi, etc.)	module/iam/ appliqué
■	Cluster EKS 1.30 créé, API privé, logging activé	module/eks/ appliqué
■	Node group stateful-ng : 1 nœud m5.xlarge actif	kubectl get nodes
■	Node group compute-ng : 0 nœud (scale-to-zero OK)	kubectl get nodes
■	9 namespaces Kubernetes créés	kubectl get ns
■	RBAC appliqué sur chaque namespace	kubectl get rolebindings -A
■	NetworkPolicies : trafic cross-namespace contrôlé	kubectl get networkpolicies -A
■	StorageClass gp3 (chiffrée, default)	kubectl get sc
■	Pipeline GitHub Actions : plan sur PR, apply sur main	Premier workflow OK
■	Billing alert AWS configuré à \$50 et \$100	console.aws.amazon.com/billing

Les 6 phases suivantes — Vue d'ensemble

Phase	Timing	Livrable principal
Phase 2	Sem. 3–4	MinIO Operator + Tenant, Hive Metastore, CloudNativePG, Cloudflare Tunnel
Phase 3	Sem. 5–6	Strimzi Kafka, Apache NiFi, premier pipeline CDC → Bronze Iceberg
Phase 4	Sem. 7–8	Spark Operator, dbt Silver, Airflow, ingestion fichiers XLSX
Phase 5	Sem. 9–10	Dremio Nautilus, Gold layer dbt, connexion Power BI
Phase 6	Sem. 11	Prometheus + Grafana, Fluent Bit, OpenObserve, Lambda scaling
Phase 7	Sem. 12	JupyterHub, ArgoCD (optionnel), performance tuning, démo finale

■ L'objectif final de la démo (Semaine 12)

Montrer en live le flux complet : une ligne insérée dans PostgreSQL → capturée par NiFi/Kafka en temps réel → transformée par Spark → nettoyée par dbt → visible dans Power BI en moins de 60 secondes. C'est ça, la preuve de valeur de tout ce travail.